

07 장 학습 과제

과제 1. 저장 기간과 소멸 순서 추적

생성자와 소멸자에서 객체 이름을 출력하는 **LifetimeTrace** 클래스를 작성한다.

저장 기간을 비교할 수 있도록 여러 위치에 객체를 선언한다.

- 이름 공간 범위의 전역 객체
- 함수의 자동 지역 객체
- 중첩 블록의 지역 객체
- 함수의 정적 지역 객체
- **thread_local** 객체
- **new**로 생성한 동적 객체

각 객체의 생성과 소멸 순서를 기록하고 저장 기간을 분류한다. 동적 객체에 **delete**를 호출하지 않았을 때 소멸자가 실행되지 않는 것도 확인한다.

서로 다른 소스 파일에 전역 객체를 하나씩 정의한 뒤 초기화 순서에 의존하면 안 되는 이유를 설명한다.

과제 2. 이동 전용 **FileHandle**

std::FILE*을 소유하는 **FileHandle** 클래스를 작성한다.

필수 기능

- 생성자에서 파일 열기
- 파일 열기에 실패하면 **std::runtime_error** 발생
- 소멸자에서 파일 닫기
- 복사 생성자와 복사 대입 연산자 삭제
- 이동 생성자와 이동 대입 연산자 제공
- 이동 함수에 **noexcept** 지정
- **Write()**와 **IsOpen()** 제공
- 이동 후 원본은 빈 상태

```
FileHandle first("log.txt", "w");
first.Write("first\n");
FileHandle second = std::move(first);
second.Write("second\n");
```

조기 반환과 예외가 발생하는 함수에서 파일이 정상적으로 닫히는지 확인한다.

과제 3. 예외 안전한 **CharacterInventory**

std::vector<std::string>으로 아이템을 관리하는 **CharacterInventory**를 작성한다.

필수 기능

```
void AddItem(const std::string& item);
void ReplaceItems( const std::vector<std::string>& items);
std::size_t ItemCount() const;
```

```
const std::string& At(std::size_t index) const;
```

At()은 범위를 벗어나면 `std::out_of_range`를 발생시킨다. `Replaceltems()`는 임시 복사본과 `swap()`을 사용하여 강한 예외 보장을 제공한다.

예외 안전성 관점에서 네 항목을 설명한다.

- `AddItem()`이 제공하는 예외 보장
- `Replaceltems()`가 실패했을 때 기존 목록이 유지되는 이유
- 아이템 개수를 별도 멤버로 중복 저장하지 않는 이유
- `swap()`에 `noexcept`가 중요한 이유

과제 4. `unique_ptr`를 사용하는 `GameScene`

`GameScene`이 여러 `Enemy`를 고유 소유하도록 작성한다.

```
class GameScene
{
private:
    std::vector<std::unique_ptr<Enemy>> enemies;
};
```

필수 기능

```
std::unique_ptr<Enemy> CreateEnemy(
    int id,
    int hp
);
```

```
void AddEnemy(std::unique_ptr<Enemy> enemy);
Enemy* FindEnemy(int id);
const Enemy* FindEnemy(int id) const;
```

`CreateEnemy()`는 새 고유 소유권을 반환하고 `AddEnemy()`는 소유권을 받는다.

`FindEnemy()`가 반환하는 원시 포인터는 비소유 포인터임을 설명한다.

`GameScene`을 복사할 수 없는 이유를 확인하고, 장면 이동을 허용할지 정책을 결정한다.

과제 5. 공유 텍스처 캐시

파일 이름으로 텍스처를 검색하는 `TextureCache`를 작성한다.

```
std::unordered_map<std::string, std::weak_ptr<Texture>> textures;
```

필수 기능

```
std::shared_ptr<Texture> Load(const std::string& fileName);
void RemoveExpired();
std::size_t EntryCount() const;
```

같은 파일을 두 번 요청하면 살아 있는 기존 텍스처를 반환해야 한다. 모든 외부 `shared_ptr`가 사라지면 텍스처가 소멸하고 캐시 항목은 만료되어야 한다.

같은 원시 포인터로 두 `shared_ptr`를 만드는 잘못된 방식도 별도 테스트 코드로 작성하고 사용하면 안 되는 이유를 설명한다.

```
Texture* raw = new Texture("hero.png");
std::shared_ptr<Texture> first(raw);
std::shared_ptr<Texture> second(raw);
```

실제로 실행하여 오류를 유발하지 말고 제어 블록 구조를 그림으로 설명한다.

과제 6. `weak_ptr`를 이용한 `SceneNode`

장면 트리의 노드를 나타내는 `SceneNode`를 작성한다.

상태

- 노드 이름
- 자식 노드 목록
- 부모 노드

소유 정책

- 부모는 자식을 `shared_ptr`로 소유
- 자식은 부모를 `weak_ptr`로 관찰

```
class SceneNode : public std::enable_shared_from_this<SceneNode>
{
private:
    std::string name;
    std::vector<std::shared_ptr<SceneNode>> children;
    std::weak_ptr<SceneNode> parent;
};
```

필수 기능

```
void AddChild(std::shared_ptr<SceneNode> child);
std::shared_ptr<SceneNode> Parent() const;
void PrintPath() const;
```

`AddChild()`에서는 `shared_from_this()`를 사용하여 자식의 부모를 설정한다. 객체가 반드시 `std::make_shared<SceneNode>()`로 생성되어야 하는 이유를 설명한다.

부모와 자식을 모두 `shared_ptr`로 만들었을 때 순환 참조가 발생하는 구조와 `weak_ptr`로 변경한 구조를 UML로 작성한다.

과제 7. `ResourceManager`의 예외 처리

리소스 이름으로 데이터를 로드하는 `ResourceManager`를 작성한다.

오류 조건

- 빈 리소스 이름: `std::invalid_argument`

- 파일이 없음: 사용자 정의 `ResourceError`
- 인덱스 범위 초과: `std::out_of_range`

호출 코드는 구체적인 예외부터 처리하고 마지막에 `std::exception`을 처리한다.

RAII 추적 객체를 함께 사용하여 예외가 발생했을 때 지역 객체가 소멸하는 순서를 출력한다. 생성자에서 두 개의 자원을 순서대로 획득하다가 두 번째 자원에서 실패하는 클래스도 작성하고, 첫 번째 자원이 자동으로 정리되는지 확인한다. 소멸자에서는 예외를 밖으로 전달하지 않는다.

과제 8. 게임 객체 소유권 설계와 UML

지정된 객체로 구성된 게임 구조를 설계한다.

- `GameApplication`
- `GameScene`
- `Player`
- `Enemy`
- `Texture`
- `Sprite`
- `AudioClip`
- `AudioPlayer`
- `TargetSystem`
- `TextureCache`

각 관계는 여섯 가지 표현 중 하나를 선택해 나타낸다.

값 멤버
참조
비소유 원시 포인터
`unique_ptr`
`shared_ptr`
`weak_ptr`

선택 이유와 대상의 수명 조건을 작성하고 주어진 요구를 설계에 반영한다.

- 애플리케이션은 현재 장면을 고유 소유
- 장면은 플레이어와 적을 소유
- 여러 스프라이트가 텍스처를 공유
- 텍스처 캐시는 사용되지 않는 텍스처 수명을 연장하지 않음
- 적은 플레이어를 공격 대상으로 관찰하지만 소유하지 않음
- 여러 오디오 재생 객체가 하나의 오디오 데이터를 공유할 수 있음

UML 클래스 다이어그램에 합성 관계와 일반 연관 관계를 표시하고 관계선에 `<<unique_ptr>>`, `<<shared_ptr>>`, `<<weak_ptr>>` 또는 `<<non-owning>>`를 기록한다. 함수 인터페이스도 소유권 의미가 드러나도록 작성한다.